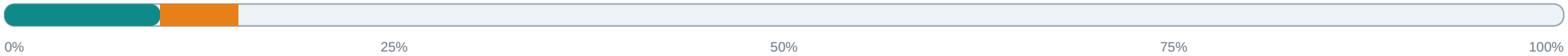
OrbitriX: Zero to 100% Roadmap

12 phases, built in this order



Done | In progress | Next | Planned

Percent in each box = total project completed when that phase is finished.

PHASE 0 — DONE

Setup tools

Install VS Code, Node.js, Git and Docker Desktop. Check each version in the terminal.

5%

PHASE 1 — DONE

Local base

Supabase CLI, local start, people table with 5 fake rows, first Git commits.

10%

PHASE 2 — IN PROGRESS

Team repo

Private GitHub repository, README for teammates, .env.example, invite teammates later.

15%

PHASE 3 — NEXT

Admission database

Zones, colleges, departments, cycles, applications, 4 levels, audit history, deadline and reason rules.

25%

PHASE 4 — PLANNED

Login, roles, RLS

Roles: applicant, volunteer, panel, coordinator, student, alumnus. Access rules for each role.

35%

PHASE 5 — PLANNED

Admission screens

Apply with drafts, tracker, staff review, bulk select or reject, selection creates the student.

50%

PHASE 6 — PLANNED

Smart uploads

Signed links, type and size checks, oversized file rejected, storage chosen by config.

60%

PHASE 7 — PLANNED

Event log and replay

Events saved with each change. Replay from start, a time or a bookmark. No duplicate emails. Late commits not missed.

75%

PHASE 8 — PLANNED

Alumni career tracker

Job history with source, company merge rules, profile built by replay, search by company.

85%

PHASE 9 — PLANNED

Student and alumni views

Dashboard with zone, college and ID. Connect me as Alumni. Zone event list. Volunteering count.

92%

PHASE 10 — PLANNED

Data, tests, one command

Synthetic data seed, tests for the tricky logic, README, one-command start, no secrets in code.

97%

No: fix it and run the checks again

All checks pass? (tests, start, demo)

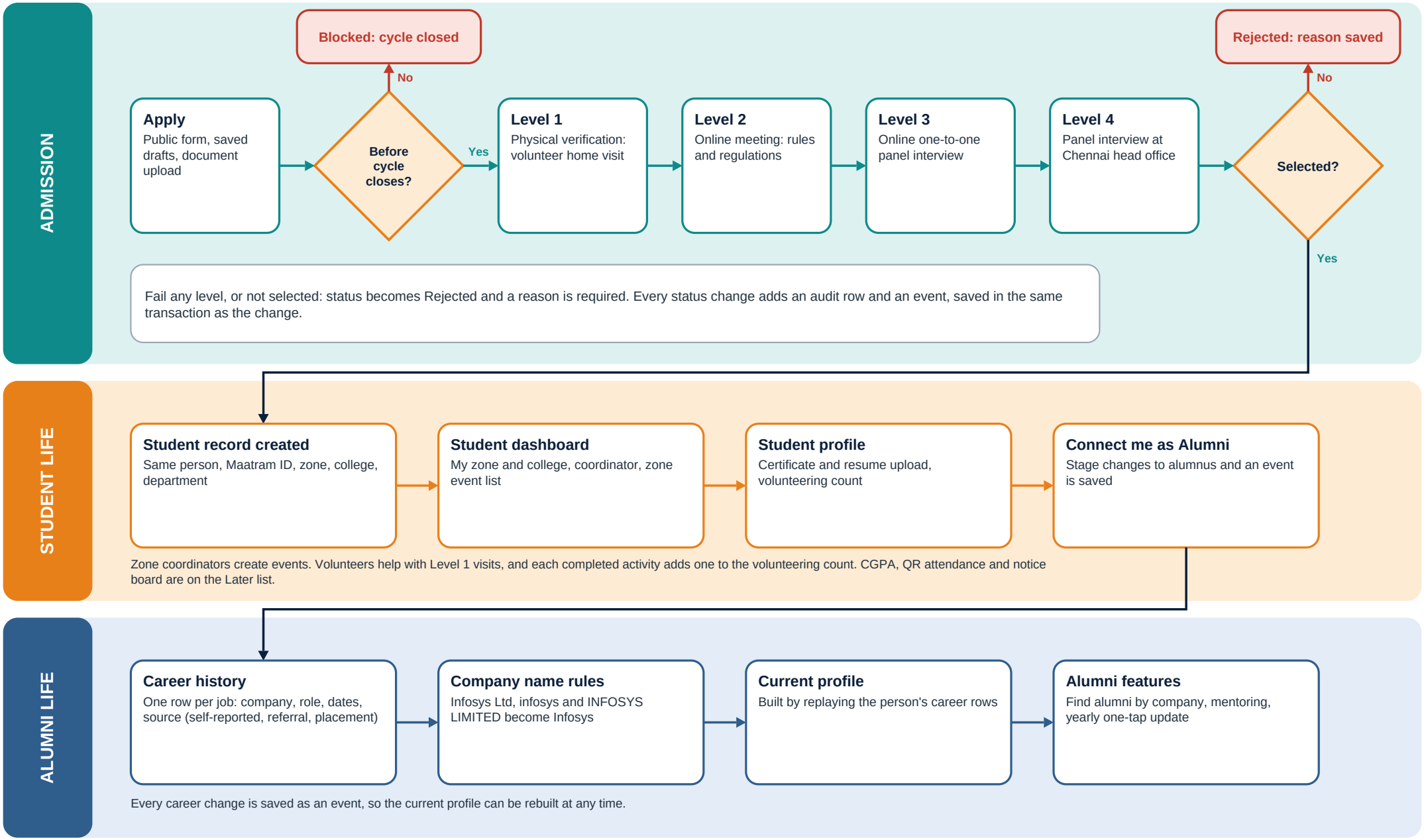
Yes

100% COMPLETE: record the demo and submit

Rule: finish a phase only when its "done when" check passes (page 5). Question the requirement, delete extras, simplify, then speed up and automate.

OrbitriX: User Journey Flow

Admission, student life, alumni life: one person, one Maatram ID



OrbitriX: Event Log and Replay Flow

Background work that is safe to repeat

A. WRITE PATH: every change leaves an event

Browser (React)
Applicant submits, staff clicks Select, alumnus adds a job

→

Supabase API
Login check and row-level security rules

→

One database transaction
Change + event row + audit row saved together, or none of them

→

events table
Append-only and numbered. Never edited, never deleted

B. REPLAY PATH: background worker

Replay worker (Node.js)
Choose a mode: from the start, from a time, or from its saved bookmark

→

Read safe events
Only events from fully finished transactions, so late commits are never skipped

→

Already sent?

No →

Do the work
Send the email, record it in the outbox, move the bookmark forward

Yes ↓

Skip: no duplicate email

Same-transaction events

The change and its event are saved in one transaction. If one fails, both are undone, so the log never disagrees with the data.

Out-of-order commits

Event numbers are given before commit, and commits can finish in a different order. The worker reads only up to the point where every earlier transaction has finished, so a late commit is picked up on the next round.

No duplicate emails

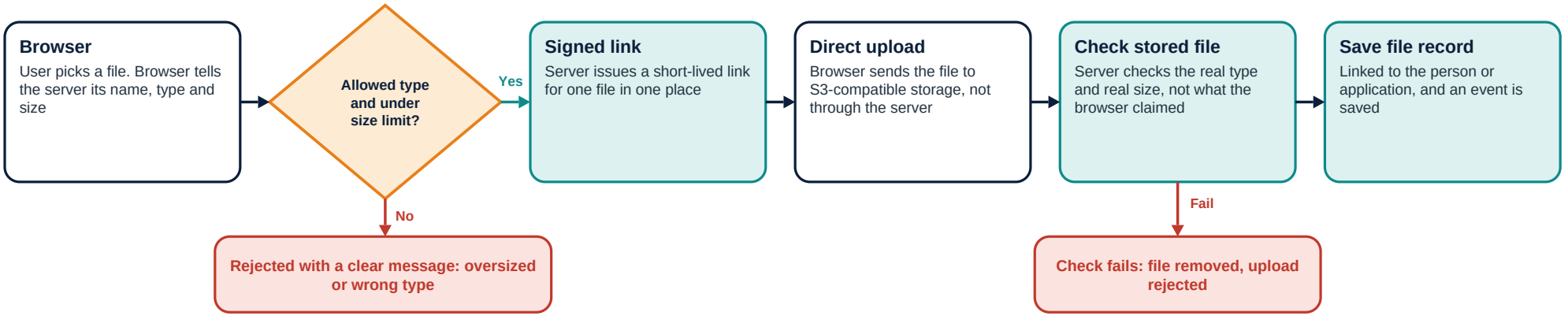
Each email is recorded in an outbox with a unique key (event, person, email type). Replaying the same event finds the key and skips sending. A repeated event with the same key is stored once.

Outages and bad input

If the worker or database stops, the worker resumes from its bookmark. An event that cannot be processed is marked failed with an error note, and the rest continue.

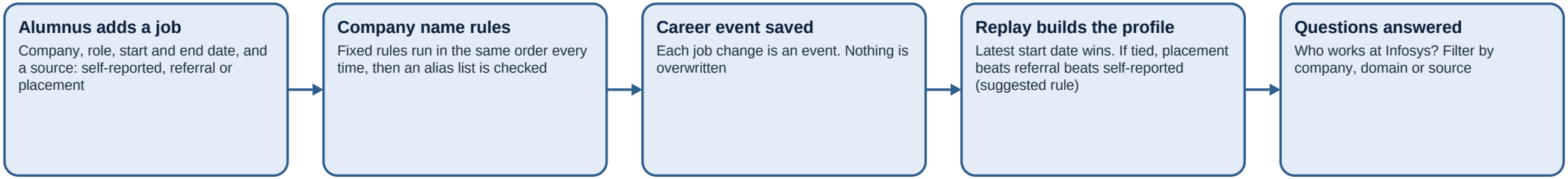
OrbitriX: Uploads and Career Tracker Flow

A. SMART UPLOADS: the file goes straight to storage, the server checks the rules

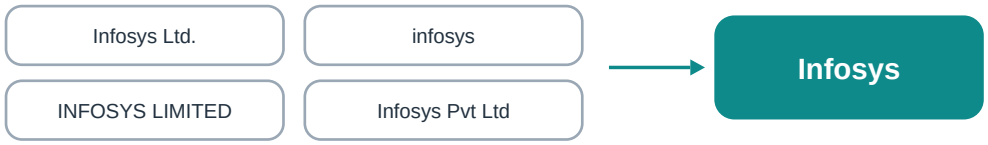


Switch storage provider: change only the .env values (endpoint, bucket, access key, secret key). The code stays the same. Locally: MinIO or Supabase Storage. Keys are never written in code.

B. ALUMNI CAREER TRACKER: history rows in, current profile out



Different spellings, same company:



Names the rules cannot join are added to the alias list by hand, once.

Company rules, always in this order

1. Make lowercase
2. Trim spaces and remove punctuation
3. Drop endings such as ltd, limited, pvt, inc, llc
4. Look up the alias list (for names that need a human decision)
5. Same input always gives the same result

Rules are written down in the README, so anyone can predict the result.

OrbitriX: Phase Checklist

Do not start a phase until its done-when check passes

Phase	What to build	Done when	Total
0. Setup tools	Install VS Code, Node.js, Git, Docker Desktop	node -v, git --version and docker --version print versions. Docker Desktop shows running.	**5%**
1. Local base	Supabase CLI, local start, people table, 5 fake rows, first commits	Studio opens at 127.0.0.1:54323. Table Editor shows 5 rows. git log shows commits.	**10%**
2. Team repo	Private GitHub repository, README for teammates, .env.example	Repository visible on GitHub. No .env or keys committed. A teammate can clone it.	**15%**
3. Admission database	zones, colleges, departments, cycles, applications, application_levels, status history, triggers, rules	Submit after close fails. Reject without reason fails. History rows appear. Deleting history fails.	**25%**
4. Login, roles, RLS	Sign-in, roles, access policies for each role	Applicant sees only own application. Volunteer sees only assigned visits. Coordinator sees own zone. Staff see all.	**35%**
5. Admission screens	Apply form with drafts, tracker, staff review, bulk select or reject with reasons, student created on selection	Apply, shortlist, select works end to end. Student with Maatram ID appears. Cycle filter and year comparison work.	**50%**
6. Smart uploads	Signed link function, bucket limits, check after upload, S3 settings in .env	Valid file uploads. Oversized and wrong-type files are rejected. Changing .env points to other storage.	**60%**
7. Event log and replay	events and outbox tables, triggers, replay worker with 3 modes and a bookmark	Replay gives no duplicate emails. Out-of-order commit is still processed. Restart resumes from bookmark.	**75%**
8. Alumni career tracker	Career rows with source, company rules and alias list, profile by replay, search	Who works at a company returns the right people across spellings. Profile matches the latest job.	**85%**
9. Student and alumni views	Dashboard, Connect me as Alumni, zone event list, volunteering count	Student sees zone and college. Button changes stage and saves an event. Count equals completed activities.	**92%**
10. Data, tests, one command	Seed script (4 zones, fake colleges, applicants, alumni), pgTAP and Vitest tests, README, start command	Fresh clone runs with one command. All tests pass. No secrets in code.	**97%**
11. Demo and submit	Record a short demo, final read-through, submit	All 6 demo scenarios shown. README is clear. Submitted.	**100%**

OrbitriX: Demo, Submission and Scope

What to show, what to hand in, what to leave for later

DEMO SCRIPT: the 6 scenarios from the problem statement

1. Replay

Replay from the start, from a time, then from the bookmark. Show that no duplicate email is sent.

2. Out-of-order events

Commit two events in reverse order. The worker still processes both.

3. Career queries

Ask who works at Infosys. Different spellings return the same people, each with its source.

4. Apply, shortlist, select

Applicant saves a draft and submits. Staff move the levels and bulk select with a reason. A student with a Maatram ID appears.

5. Cycle filtering

Switch between 2025 and 2026. Show the year-over-year comparison and the audit trail.

6. Oversized upload

A valid file uploads. An oversized file is rejected with a clear message.

SUBMISSION CHECKLIST

- [ ] Runs locally with one command (written in the README)
- [ ] No secrets in code: only .env.example is committed
- [ ] Synthetic (fake) data only, no real students
- [ ] Tests for replay, out-of-order events, company rules, cycle deadline and uploads
- [ ] Edge cases handled: duplicates, outages, bad input, oversized files
- [ ] README: setup, architecture, demo steps, assumptions
- [ ] Short demo video covering the 6 scenarios
- [ ] Teammates added to the GitHub repository

SCOPE RULE (your 5-step process)

Must — The 4 required parts, zones, 4 levels, roles, simple student dashboard, Connect me as Alumni.

Should — Zone event list, volunteering count, resume upload.

Later — QR attendance, CGPA, interview counts, mentoring, CTC, notice board.

Order never changes:

1. Question every requirement, even ones from smart people.
2. Delete what is not needed. If nothing comes back, delete more.
3. Simplify. Do not polish something that should not exist.
4. Speed up only after steps 1 to 3.
5. Automate last: seed script, one-command start, tests.

Risks to watch: the replay worker is the hardest part, Docker needs about 4 GB of RAM on an 8 GB laptop (close CAD and other heavy apps), and a solo build needs small steps with a commit after each one.